CHAPTER – 02

Question 1: What is the purpose of system calls?

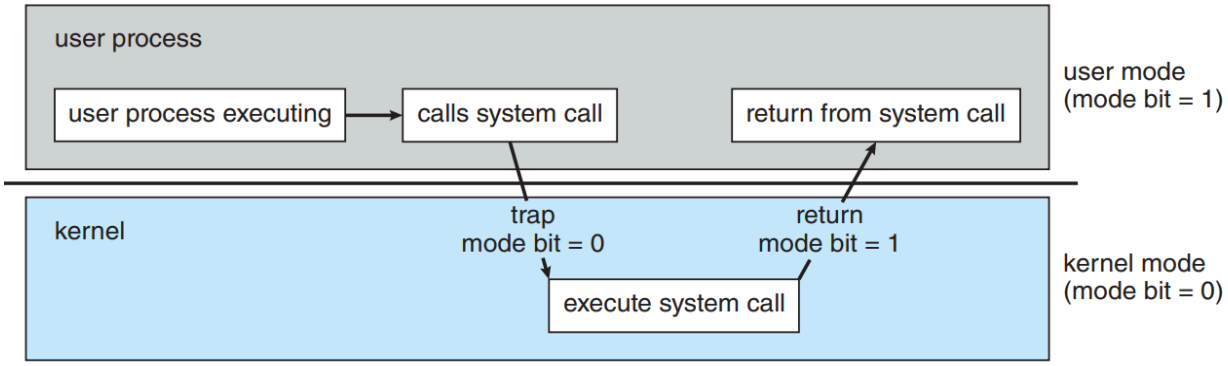


Figure 1.13 Transition from user to kernel mode.

The purpose of system calls is to provide an interface between user-level applications and the operating system. System calls enable user programs to request services from the operating system's kernel.

- **Resource Access**: Allow programs to request access to hardware resources like memory, files, and devices.
- **Security**: Enforce protection by controlling and limiting access to critical system functions.
- **Abstraction**: Provide a simplified interface for interacting with hardware, abstracting low-level details.
- **Stability**: Ensure safe execution of programs without compromising system stability.
- **Convenience**: Simplify software development by handling complex operations like I/O and process management within the OS.

Question 2: What is the purpose of the command interpreter? Why is it usually separate from the kernel?

Purpose of the Command Interpreter:

1. **User Interface**:

- Acts as a link between you and the operating system, letting you enter commands to control the system.
- ***Example*:** You type `ls` to list files in a directory. The interpreter processes this and shows you the files.

2. **Reading and Interpreting Commands**:

- Reads your commands and carries out the necessary actions.
- ***Example:*** You type `rm file.txt` to remove a file. The interpreter interprets this and remove `file.txt`.

3. **Executing System Programs**:

- Runs programs by converting your commands into system actions.
- ***Example*****:** You type `python os.py` to run a Python script. The interpreter starts Python and runs `os.py`

Reasons why the command interpreter is usually separate from the kernel:

1. **Modularity and Flexibility**: Users can replace or customize the command interpreter without altering the kernel.
2. **Security**: Impact of interpreter crashes to user space, protecting the kernel.
3. **Stability**: A kernel reduces complexity and potential bugs, enhancing overall system reliability.
4. **Resource Management**: The kernel can manage resources more efficiently without the excess load of user-interface processes.
5. **Ease of Development**: Independent development simplifies updates and maintenance for both the kernel and the interpreter.
6. **User-Kernel Separation**: Clear boundaries via system calls maintain system integrity and prevent unauthorized access to kernel functions.

Question 3: What system calls have to be executed by a command interpreter or shell in order to start a new process on a UNIX system?

In a UNIX system, to start a new process, the shell uses two main system calls:

1. **`fork()`**: This system call creates a new process, called the child process, by duplicating the current (parent) process. The child process is a copy of the parent and is used to execute a new program.
2. **`exec()`**: After `fork()`, the child process uses `exec()` to replace itself with the new program (e.g., `ls`). This system call loads and runs the new program in the child's memory space.

Optionally, the parent process may use **`wait()`** to wait for the child process to finish before continuing its execution.

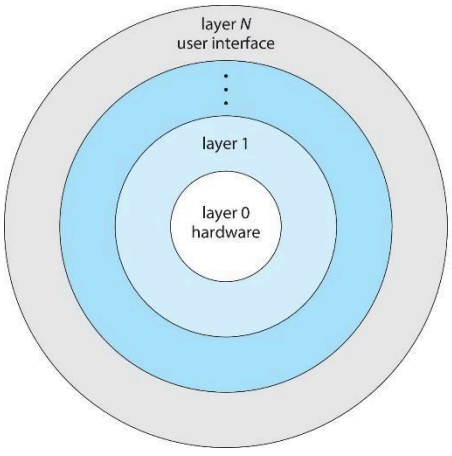
fork() **creates a new process**, and **exec()** **runs the new program** in that process. In real example, you as the user only see the result (Documents, Downloads, etc.), but internally the shell is using fork() and exec() to manage the processes that handle your command.

Question 4: What is the purpose of system programs?

System programs are executed at the **operating system** and interact with the **kernel** to perform various tasks related to system management. It acts as an intermediary between the user, the applications, and the computer's hardware.

1. **Resource Management**: They allocate and manage the computer's hardware resources, such as CPU time, memory, storage, and I/O devices.
2. **File Management**: They manage files, directories, and storage devices, ensuring that data is stored and retrieved properly.
3. **Process Management**: They schedule and control the execution of multiple processes, including task switching and inter-process communication.
4. **Security and Access Control**: They enforce security policies by managing user permissions and preventing unauthorized access to system resources.
5. **Device Management**: They facilitate communication between the system and external devices (e.g., printers, scanners, and USB devices).
6. **Error Handling**: System programs monitor the system's performance and handle errors to ensure that the system runs smoothly and recovers from failures efficiently.
7. **User Interface**: Some system programs provide interfaces, like command-line or graphical user interfaces (GUIs), allowing users to interact with the system.

Question 5: What is the main advantage of the layered approach to system design? What are the disadvantages of the layered approach?



Main Advantage of the Layered Approach:

The **layered approach** to system design divides the operating system into different layers, each performing specific functions. The **main advantage** is **modularity**, which provides the following benefits:

1. **Simplified Debugging and Maintenance**: Each layer can be developed and tested independently. Issues can be isolated and addressed in specific layers without affecting the entire system.
2. **Abstraction**: Higher layers use services from lower layers without needing to know how they work. This abstraction simplifies development and makes the system more flexible and maintainable.

Disadvantages of the Layered Approach:

- **Performance Overhead**: Each layer must communicate with the one below it, which can slow down the system due to extra communication.
- **Layer Dependency**: If one layer relies too much on another, changes in lower layers may affect higher ones, causing rework.
- **Complexity in Layering**: Designing an efficient layered system can be tricky, especially when deciding which functions go into which layers and minimizing interaction between them.

Question 6: List five services provided by an operating system, and explain how each creates convenience for users.

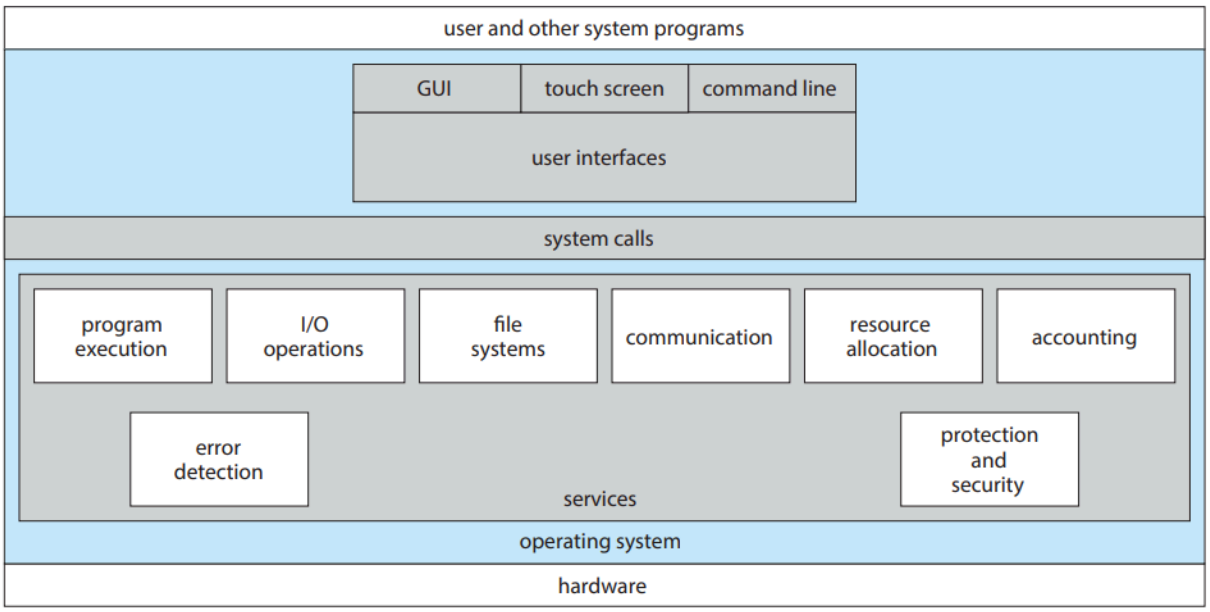


Figure 2.1 A view of operating system services.

1. **Program Execution**: When you want to run an app or software, the system program automatically loads it into memory and makes sure it runs smoothly. You don't need to do anything manually, it handles everything behind the scenes.
2. **I/O Operations**: "I/O" means communication between your computer and external devices (like a keyboard, mouse, printer, or USB drive). The system program manages this communication so you can easily use these devices without having to worry about how they actually work.
3. **File System Manipulation**: This makes it easy for you to organize and manage files on your computer. Whether you want to create, delete, move, or rename a file, system programs handle these tasks for you, so you can store and retrieve your data effortlessly.
4. **Communications**: Computers often have multiple programs running at the same time. System programs help these applications communicate with each other. For example, when one program sends data to another, system programs ensure the transfer happens correctly, enabling collaboration between applications.
5. **Error Detection**: System programs monitor the health of your computer by checking for problems in hardware (like a malfunctioning hard drive) or software (like a bug in an app). When errors are found, they try to fix them or alert you, helping keep your system stable and secure.

Question 7: Why do some systems store the operating system in firmware, while others store it on disk?

The decision to store an operating system in firmware or on a disk depends on factors like usage, performance, hardware limits, and security. Here's a quick overview:

Storing OS in Firmware

Advantages: Quick boot times, enhanced security through read-only setup, higher reliability with no moving parts, and lower cost and complexity for devices with limited functionality.

Use Cases: Ideal for embedded systems such as automotive controls or IoT devices where speed, security, and reliability are crucial.

Storing OS on Disk

Advantages: Greater flexibility for upgrades, higher storage capacity, cost-effectiveness for large storage needs, and potentially better performance with modern SSDs.

Use Cases: Best suited for general-purpose computers and servers that require regular updates and have extensive storage needs.

Question 8: How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?

To design a system that allows booting multiple operating systems, you would typically use a boot manager or bootloader. Here's a concise breakdown:

System Design

1**. Partitioning**: Create separate disk partitions for each OS, ensuring each has its own space for system files.

2. **Bootloader Installation**: Install a bootloader like GRUB in the Master Boot Record (MBR) or GUID Partition Table (GPT) that supports multiple OS options.

3. **Configuration**: Set up the bootloader's configuration file to list all available operating systems with paths to their kernels and necessary boot parameters

Bootstrap Program Tasks

1. **Initialization**: Check and prepare system hardware for booting.

2. **Menu Display**: Show a menu for OS selection based on configured options.

3. **OS Loading**: Load the selected OS kernel into memory and hand over control.

4. **Chain Loading:** Manage the loading of secondary bootloaders if required (e.g., Windows Boot Manager).

5. **Error Handling**: Provide diagnostics and recovery options in case of boot failures.

This approach allows users to select their desired operating system at startup, catering to different needs and preferences within a single machine.

Question 9: The services and functions provided by an operating system can be divided into two main categories. Briefly describe the two categories, and discuss how they differ.

The services and functions provided by an operating system (OS) can be broadly categorized into two main types: User Services and System Services. Each category plays a crucial role in the functioning of a computer, but they serve different purposes and cater to different audiences.

User Services

User services are designed to make the computer system convenient and efficient for the user. These include:

User Interface (UI): Can be graphical (GUI) or command-line (CLI). This service provides the means for user interaction with the computer system.

Program Execution: The OS manages the execution of user programs, providing services like task scheduling, memory allocation, and handling program inputs and outputs.

File Management: Allows users to create, delete, read, write, and manipulate files in a structured and secure way.

Device Communication: Manages communication between the user programs and hardware devices using drivers and APIs.

User services are primarily about ease of use, accessibility, and providing a seamless and responsive computing experience.

System Services

System services, on the other hand, are concerned with managing the computer hardware and software resources. These services are mostly invisible to the user and include:

Resource Management: The OS optimally allocates resources like CPU time, memory space, and device access to ensure efficient system performance.

Security and Access Control: Protects information and system resources against unauthorized access through user authentication, data encryption, and access permissions.

Error Detection and Handling: Monitors system and application software to detect and respond to errors, which helps in maintaining system stability and security.

Job Accounting: Keeps track of resource usage by different jobs and users for system monitoring and improving resource allocation.

System services ensure the stable and efficient operation of the system, focusing on performance, resource management, and security.

How They Differ

Target Audience: User services are directly utilized by the end-user, whereas system services run in the background and are typically only accessed or noticed by system administrators.

Goal: The primary goal of user services is usability and user satisfaction, while system services aim at efficiency, stability, and security of the system operations.

Interaction Level: User services interact more frequently and directly with the user, while system services operate transparently, providing foundational support for user services and applications

Both categories are essential, working together to provide a robust, user-friendly, and secure computing environment.

Question 10: Describe three general methods for passing parameters to the operating system.

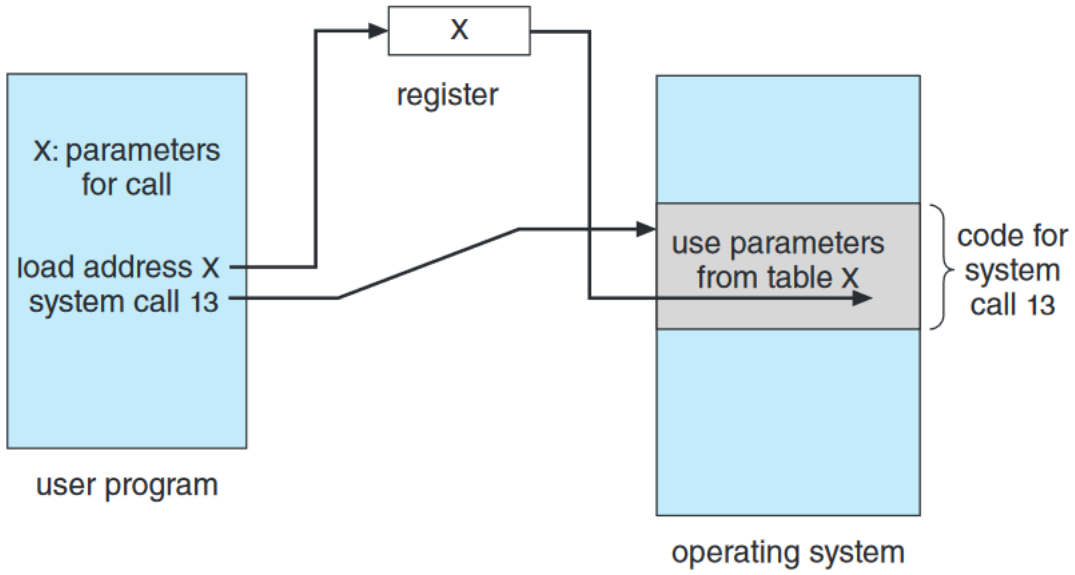


Figure 2.7 Passing of parameters as a table.

Parameters can be passed to the operating system (OS) in several ways to control the behavior of system calls and services. Here are three general methods:

1. **Through Registers**

Description: The simplest and fastest method for passing small amounts of data. Parameters for system calls are placed directly into registers before the system call is made. Each register can hold a piece of data, like a pointer or a value.

Usage: Commonly used for passing fewer parameters that can fit into the available registers of the processor. It's efficient for simple system calls in many operating systems.

2. **Via a Block or Table in Memory**

Description: For system calls requiring more parameters or larger amounts of data, parameters are stored in a table or block of memory. The address of this block is then passed to the OS via a register.

Usage: This method is utilized for complex operations requiring multiple parameters, such as file operations or process creation, where parameters include file paths, access permissions, or multiple attributes.

3. **Using a Stack**

Description: Parameters can also be passed via the stack. This method pushes the parameters onto the stack in the user space before the system call, and they are popped off the stack in the kernel.

Usage: This approach is suited for systems where the stack is easily manipulated and can be more flexible than using registers, especially if the number or size of parameters is not fixed or too large for the registers.

Each method has its use case, determined by factors like the speed of access, the number of parameters, and the complexity of the system call. The choice depends on the design and requirements of the operating system as well as the hardware architecture**.**

Question 11: Describe how you could obtain a statistical profile of the amount of time a program spends executing different sections of its code. Discuss the importance of obtaining such a statistical profile.

Profiling is a method used to analyze how a program spends time across different code sections, essential for optimizing performance. Here's a succinct overview:

Methods to Obtain a Statistical Profile

1. Instrumentation: Manually or automatically insert code to record execution times.

2. Sampling: Use tools like "gprof" or "perf" or "Visual Studio Profiler to periodically record the active code segment during program execution.

Importance of Profiling

- **Performance Optimization**: Identifies slow sections for targeted improvements.

- **Resource Management**: Aids in efficient resource use in complex environments.

- **Cost Efficiency**: Reduces operational costs by streamlining code.

- Quality Assurance: Helps maintain performance standards and track regressions.

Profiling tools typically generate visual reports (e.g., pie charts, bar graphs, flame graphs) that help developers visualize time distribution, aiding in effective optimization.

Question 12: What are the advantages and disadvantages of using the same system-call interface for manipulating both files and devices?

Using the same system-call interface for manipulating both files and devices can streamline programming and system design, but it also presents certain challenges. Here's a brief overview of the advantages and disadvantages:

Advantages

1. **Simplicity**: Reduces complexity in coding, making the API easier for developers to learn and use.
2. **Consistency**: Ensures a uniform set of operations across various types of resources, enhancing code maintainability.
3. **Portability**: Improves software portability across different systems by abstracting the handling differences between files and devices.

Disadvantages

1. **Performance Issues**: May lead to inefficiencies since devices might need more specific interactions than files allow.
2. **Complexity in System Design**: Simplifies the API at the cost of complicating the OS's internal mechanisms to accommodate both files and device specifics.
3. **Security Concerns**: Uniform access methods can expose the system to security vulnerabilities due to inadequate device-specific access controls.

This approach, while streamlining development, necessitates careful management of performance and security aspects in complex computing environments.

Question 13: Would it be possible for the user to develop a new command interpreter using the system-call interface provided by the operating system?

Yes, it's possible for a user to develop a new command interpreter (or shell) using the system-call interface provided by the operating system. The system-call interface allows programs to request services from the OS, such as creating processes, handling input/output, and managing files. By using these system calls, you can build a custom shell that interacts with the operating system to execute user commands.

Steps Involved:

1. **Reading User Input**: The interpreter reads commands entered by the user through the console or terminal.

2. **Parsing Commands**: The shell then interprets the input by breaking it into a command and its arguments.
3. **Executing Commands**: Using system calls like `fork()` (to create a new process) and `exec()` (to run a program), the interpreter can execute external programs or built-in commands.
4. **Handling I/O**: System calls like `read()`, `write()`, and `open()` manage file operations and input/output, allowing the shell to handle file manipulations, redirection, and device communication.
5. **Error Handling**: The shell uses system calls to detect and report errors when commands fail or resources are unavailable.

By utilizing these system calls, you can create a custom command interpreter with new features or specialized functionality. It gives flexibility and control, but also requires careful handling of user input and system resources to ensure security and stability.

Question 14: Describe why Android uses ahead-of-time (AOT) rather than just-in-time (JIT) compilation.

1. **Optimized Resource Use**: AOT compiles app code during installation, reducing CPU and memory usage during runtime.
2. **Improved Battery Life**: By avoiding runtime compilation, AOT helps conserve battery power.
3. **Faster App Launch**: Pre-compiling code allows apps to start more quickly without delays.
4. **Lower Memory Usage**: AOT reduces the need for storing both bytecode and compiled code, saving memory space.

Question 15: What are the two models of inter-process communication? What are the strengths and weaknesses of the two approaches?

1. Message-Passing Model:

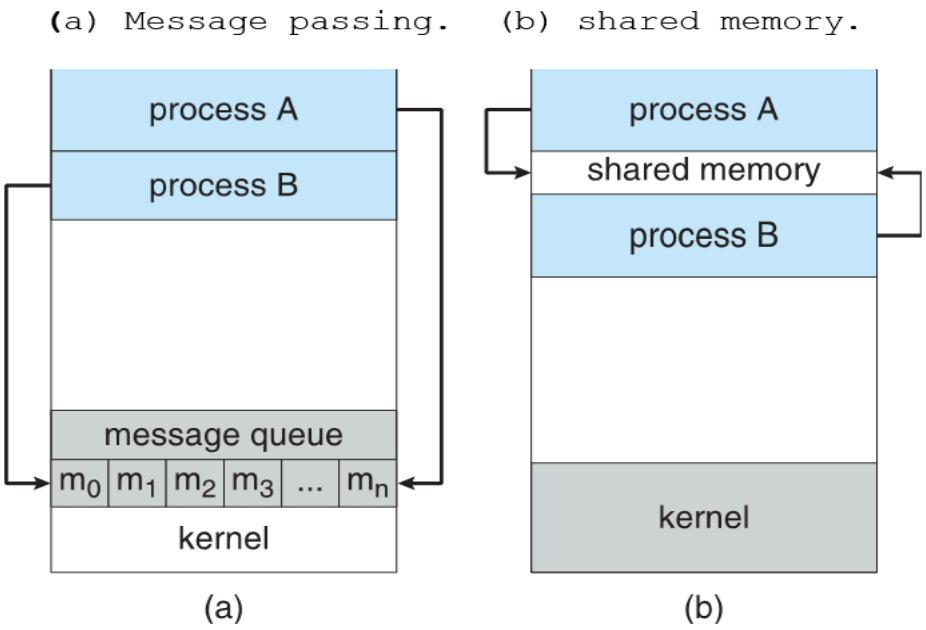
In this model, processes communicate by sending and receiving messages. Communication can happen either directly between processes or indirectly through a common intermediary, such as a mailbox or message queue.

Strengths:

- **Easier to Implement**: It's simpler to implement, especially for communication across different computers, since the OS manages the message transfer.
- **No Synchronization Issues**: There's no need to worry about processes accessing shared memory simultaneously, reducing complexity.
- **Safer for Small Data**: It's suitable for small data transfers without the risk of memory conflicts.

Weaknesses:

- **Slower for Large Data**: The overhead of sending and receiving messages can slow down communication, especially with large amounts of data.
- **Limited by OS**: The speed and efficiency depend on the operating system's message-passing implementation, and it may involve more system calls, increasing overhead.



2. Shared-Memory Model:

In this model, two or more processes share a region of memory. They exchange information by reading from and writing to this shared memory space, allowing faster communication within the same system.

Strengths:

- **High Speed**: Communication happens at memory transfer speeds, making it very fast, especially for large data transfers within the same computer.
- **Efficient for Large Data**: Since the data is directly shared, it avoids the overhead of copying or sending messages.

Weaknesses:

- **Complex Synchronization**: The processes must manage access to the shared memory to avoid conflicts, like writing to the same memory location simultaneously, which requires synchronization mechanisms.
- **Security and Protection**: Shared memory opens up risks of data corruption or security vulnerabilities if not managed carefully, as processes can potentially access each other's memory space.

Question 16: Contrast and compare an application programming interface (API) and an application binary interface (ABI).

Aspect	API (Application Programming Interface)	ABI (Application Binary Interface)
Definition	A set of functions that allows programs to communicate with each other.	Rules that allow programs to run on specific hardware.
Level	Works with **source code** (before compiling).	Works with **machine code** (after compiling).
Portability	Can be used across different platforms.	Depends on the specific hardware and operating system.
Changes Impact	Changes require modifying the program code.	Changes require recompiling the program.
Example	Using a library like `stdio.h` in C for input/output.	The system's rules for running an executable file.
Focus	Focuses on how programs interact with other software.	Focuses on how programs interact with hardware and the OS.

Question 17: Why is the separation of mechanism and policy desirable?

The separation of **mechanism** and **policy** (what is done) is desirable because it offers several benefits:

1. **Flexibility**: By separating the two, system designers can use the same mechanisms to support different policies, allowing the system to adapt to various requirements without needing to change the core mechanisms.
2. **Modularity**: This separation makes the system more modular, enabling easier maintenance and updates. Policies can be changed or adjusted without altering the underlying mechanism.
3. **Customization**: Different environments may need different policies. Separating mechanism and policy allows administrators or users to implement custom policies for specific needs, without changing how the system functions.

In summary, this separation provides a flexible, modular, and customizable system that can adapt to different requirements while maintaining the same foundational structure.

Question 18: It is sometimes difficult to achieve a layered approach if two components of the operating system are dependent on each other. Identify a scenario in which it is unclear how to layer two system components that require tight coupling of their functionalities.

A scenario where it is difficult to achieve a layered approach due to tight coupling between two components is the relationship between **memory management** and **process scheduling**.

Scenario:

Memory management is responsible for allocating and managing memory for processes, while **process scheduling** decides which process runs on the CPU at a given time.

These two components are closely dependent on each other. The process scheduler needs to know which processes have enough memory allocated to them in order to run, and the memory manager needs to know which processes are active or scheduled to ensure they have sufficient memory.

Why It's Difficult to Layer:

1. **Mutual Dependence**: The scheduler may need to pause a process if there is insufficient memory, while the memory manager needs information from the scheduler about which processes are running or waiting.
2. **Tight Integration**: Both components need constant communication and coordination, making it hard to place them in separate layers where one strictly depends on the other.

This mutual dependency creates a situation where it is unclear how to layer these components effectively, as they rely on each other's functionality to operate smoothly. In such cases, a more integrated approach is often required instead of a strictly layered structure.

Question 19: What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

Main advantage of microkernel approach

Increased Reliability and Security: By keeping only essential services in the kernel, other system services run in user space. This separation ensures that failures in non-essential components don't affect the entire system, enhancing fault isolation.

Interaction between user programs and system services in a microkernel architecture:

Message Passing: User programs interact with system services through message passing. When a user program needs a service (like file system or networking), it sends a request to the microkernel, which forwards it to the appropriate user-space service. The service processes the request and sends the result back via the microkernel. This inter-process communication (IPC) is key to maintaining system modularity and separation.

Disadvantages of the microkernel approach:

1. **Performance Overhead**: The frequent use of message passing between user programs and services can introduce significant communication overhead, leading to slower performance compared to monolithic kernels, where system calls can be handled directly.
2. **Complexity of Implementation**: Managing IPC and ensuring proper communication between many small components can increase the complexity of the overall system, making development and debugging more challenging.
3. **Higher System Resource Usage**: Since more system services run as separate user processes, a microkernel system may use more memory and CPU resources compared to a monolithic kernel, where many services are integrated directly into the kernel space.

Question 20: What are the advantages of using loadable kernel modules?

Advantages of Loadable Kernel Modules (LKMs):

1**. Modularity:** Modules can be loaded/unloaded dynamically, simplifying functionality management without needing system reboots.

2. **Flexibility:** Additional features like drivers or file systems can be added without modifying or recompiling the core kernel.

3. **Efficient Resource Usage**: Only required modules are loaded, reducing memory usage and optimizing system resources.

4. **Easier Maintenance and Updates:** Bug fixes and updates can be applied to modules without restarting the system, reducing downtime.

5. **Enhanced Security:** The attack surface is minimized by only loading essential features, unloading potentially vulnerable code when unnecessary.

6. **Development Convenience**: Easier testing and debugging of kernel code without requiring a complete kernel rebuild and reboot.

Question 21: How are iOS and Android similar? How are they different?

Similarities between iOS and Android:

1. **Mobile Operating Systems:** Both iOS and Android are designed for mobile devices like smartphones and tablets.

2. **App-Based Ecosystem:** Both rely on app stores (Apple App Store for iOS and Google Play Store for Android) with millions of apps.

3. **Touchscreen Interfaces**: Both use touchscreen navigation with similar gestures (e.g., swiping, tapping, pinching).

4. **Core Mobile Features**: Both offer essential functions like calling, messaging, internet browsing, email, GPS, and camera integration.

5. **Security Features:** Both provide encryption, biometric authentication (e.g., Face ID, fingerprint), and regular security updates.

6. **App Development Support**: Both platforms support app development via SDKs (Xcode for iOS, Android Studio for Android) and have large developer communities.

Differences between iOS and Android:

Aspect	iOS	Android
Developer	Developed by Apple.	Developed by Google.
Devices	Runs only on Apple devices like iPhone, iPad.	Runs on a wide variety of devices from different manufacturers (Samsung, OnePlus, etc.).
App Store	Uses the Apple App Store for apps.	Uses the Google Play Store for apps.
Customization	Limited customization options.	Highly customizable, with more freedom for users.
Source Code	Closed-source, proprietary.	Open-source (based on Linux), with customization options for manufacturers.

Aspect	iOS	Android
Voice Assistant	Uses Siri.	Uses Google Assistant.
Updates	Regular updates for all supported devices.	Updates depend on the manufacturer and carrier.
User Interface (UI)	Known for a simple, consistent interface.	Offers a more customizable interface with various themes and layouts.

Question 22: Explain why Java programs running on Android systems do not use the standard Java API and virtual machine.

Java programs on Android do not use the standard Java API and JVM because:

1. Dalvik/ART Runtime: Android uses its own runtime (Dalvik or Android Runtime - ART), optimized for mobile devices with limited resources like memory and battery, unlike the standard JVM.

2. Different APIs: Android provides its own APIs designed for mobile-specific features (e.g., UI, GPS, touch gestures) instead of the standard Java APIs like (javax.swing) or (java.awt).

3. Application Packaging: Android apps are compiled into DEX (Dalvik Executable) bytecode, which runs on Dalvik/ART, not the JVM's standard bytecode.

4. Performance Optimization: Dalvik/ART are optimized for mobile hardware, handling memory and CPU constraints better than the JVM, which is designed for desktops and servers.

Question 23: The experimental Synthesis operating system has an assembler incorporated in the kernel. Discuss the pros and cons of the Synthesis approach to kernel design and system performance optimization.

Pros of the Synthesis Approach:

1. Performance Optimization: By incorporating an assembler in the kernel, Synthesis can dynamically generate highly optimized code for specific tasks, leading to faster execution and improved system performance.

2. Customization: The kernel can optimize itself for the hardware and specific workloads at runtime, making it highly adaptable to varying system demands.

3. Efficient Resource Utilization: With real-time code generation, the system can reduce unnecessary overhead, improving resource utilization and response times.

Cons of the Synthesis Approach:

1. Increased Complexity: Incorporating an assembler in the kernel adds complexity to the system, making it harder to develop, maintain, and debug.

2. Security Risks: Dynamically generating code at the kernel level increases the risk of introducing vulnerabilities or bugs, which could lead to system crashes or security breaches.

3. Overhead for Code Generation: The process of generating and optimizing code dynamically could introduce overhead, which might negate some performance gains in specific scenarios.